

Eliciting Reasoning in LLMs using Logprob-based Rewards

Jérémy Barghorn, Théo Schifferli, Aymeric De Chillaz
Lindsay Bordier, Zeineb Mellouli
Supervised by Auguste Poiroux, NLP Lab
EPFL, Switzerland

Abstract

We explore inference-time reinforcement learning to elicit reasoning in language models using Group Relative Policy Optimization (GRPO) with rule-based and logprob-based rewards. Our approach includes a custom loss masking strategy to align training with reasoning-only rewards. Evaluated on math and poetry tasks, logprob rewards improve reasoning plausibility and generalization. These results highlight the potential of logprob signals as domain-agnostic rewards for structured and creative reasoning.

1 Introduction

Recent advancements in large language models (LLMs) have predominantly relied on scale, either in model size or dataset size, to improve downstream performance across a wide variety of tasks. While these trends have yielded strong gains in language understanding, reasoning capabilities remain a challenge. Techniques such as Chain-of-Thought (CoT) prompting and fine-tuning with human-annotated rationales have demonstrated some effectiveness (Wei et al., 2023; Kojima et al., 2023; Wang et al., 2023); however, they remain brittle across domains and expensive to scale.

In this work, we explore a complementary direction: enhancing LLM reasoning at inference time through reinforcement learning, rather than relying solely on architectural scaling or supervised fine-tuning. Specifically, we build upon the DeepSeek-R1 framework and its Group Relative Policy Optimization (GRPO) algorithm (DeepSeek-AI et al., 2025), incorporating log-probability-based rewards as a means of incentivizing self-consistent and generalizable reasoning behaviors.

Our central hypothesis is that rewarding a model based on its internal confidence, measured via log-probabilities assigned to a gold answer, given its own reasoning trace, can serve as a general-purpose, task-agnostic supervision signal. In con-

trast to rule-based reward systems that require hand-crafted evaluators for each domain, logprob-based rewards provide continuous feedback, enabling applicability across diverse and subjective tasks.

To evaluate this hypothesis, we conduct experiments on two different reasoning tasks: a structured symbolic math task and a creative poetry completion task. We compare traditional rule-based rewards with our proposed logprob-based reward mechanism. To mitigate verbosity bias and sequence-length effects, we apply normalization strategies *exclusively to logprob-based rewards*, including length normalization, z-score scaling, and min-max rescaling. These adjustments significantly stabilized training, especially in creative tasks like poetry generation.

This report details our methodological choices, implementation strategies, empirical findings, and provides critical discussion on the limitations and future directions of reasoning elicitation through logprob-driven reinforcement learning.

2 Related Work

2.1 Reasoning in Language Models

Much of the recent progress in LLMs has focused on reasoning capabilities, particularly through techniques such as Chain-of-Thought (CoT) prompting (Wei et al., 2023) and self-consistency decoding (Wang et al., 2023). While these methods enhance reasoning through explicit stepwise outputs, they often struggle with generalization and require domain-specific supervision.

Our work builds on this line of research by exploring reinforcement learning methods that elicit reasoning without explicit step-by-step supervision or task-specific heuristics.

Recent efforts such as DeepSeekMath (Shao et al., 2024) have also explored mathematical reasoning with LLMs at scale, combining CoT techniques with GRPO-based fine-tuning to push the

limits of symbolic problem-solving.

2.2 Reinforcement Learning with LLMs

Reinforcement learning has been widely adopted to align LLM outputs with human preferences, notably through Reinforcement Learning from Human Feedback (RLHF) using Proximal Policy Optimization (PPO) (Ouyang et al., 2022). More recently, DeepSeek introduced Group Relative Policy Optimization (GRPO) (DeepSeek-AI et al., 2025), a strategy for enhancing reasoning quality by comparing multiple completions per prompt and computing relative advantages. Kimi K1.5 builds on this idea by scaling GRPO-style reinforcement learning to larger models and longer contexts, further demonstrating the scalability of RL for inference-time reasoning (Team et al., 2025).

2.3 Logprob-Based Rewards

The notion of using log-probabilities as a reward signal has gained traction due to its domain generality and continuous nature. Instead of measuring performance via task-specific correctness, the model’s reward is derived from how likely it deems a known-correct answer, given its own reasoning.

Prior work supports this paradigm. Hao et al. (2023) propose Reasoning via Planning (RAP), where logprob-based intrinsic rewards guide Monte Carlo Tree Search to build coherent reasoning paths. Chen et al. (2024) introduce LaTRO, which optimizes latent reasoning traces via self-rewarding using the likelihood of correct answers. In creative domains, Gurung and Lapata (2025) show that logprob improvement signals can guide models to generate more coherent long-form stories.

Our work builds on these insights by applying logprob-based rewards within the GRPO framework and comparing them to rule-based variants across structured (math) and open-ended (poetry) tasks, while also exploring reward normalization for stability.

3 Methodology

The primary goal of this project was to explore strategies for improving the reasoning capabilities of small language models (with fewer than 2 billion parameters). Our focus was on eliciting explicit reasoning traces within the model’s outputs to enable more transparent and reliable evaluation. To achieve this, we designed outputs that include clear intermediate reasoning steps, such as Chain-

of-Thought (CoT) reasoning—followed by a distinct result section, where the model is assessed based on the final answer derived from its reasoning process.

To achieve this, we explored the use of GRPO guided by rule-based and logprob-based reward functions. Additionally, we examined the effects of various normalization techniques applied to the logprob-based rewards, with the goal of understanding their impact on model performance.

The effectiveness of these approaches was evaluated across two distinct tasks: mathematical problem solving and poetry generation, each chosen to probe different aspects of reasoning and creativity.

3.1 Task Definitions

We evaluate reasoning via two tasks:

- **Math Task:** Given a set of numbers and a target value, the model must construct a valid arithmetic expression that uses each number exactly once to reach the target.
- **Poetry Task:** The model is prompted with the beginning of a poem, the original author, title and form and must generate an alternative completion that adheres to these criteria while preserving semantic and structural qualities.

Each model output must follow a structured format to enable consistent evaluation of reasoning performance and allow automated verification of the final answer. This structure is essential, as rule-based reward mechanisms rely on parsing the content of the generation to determine correctness. Without a standardized format, these rules cannot reliably assess or reward the model’s outputs. Therefore, enforcing a specific output structure was a necessary step to ensure accurate and fair performance evaluation.

```
<think> Reasoning steps </think>
<answer> Final answer </answer>
```

3.2 Reward Structures

3.2.1 Rule-Based Rewards

Rule-based rewards are constructed using explicit correctness criteria specific to each task (see Section 3.1). For the math task, we evaluate properties such as valid syntax, correct result, and proper use of numbers. For the poetry task, we assess rhyme, syllable count, poetic form, and semantic similarity to a reference.

While these metrics are interpretable, they are limited in scope, highly task-dependent, and often fail to produce continuous scores. Although they can be applied to certain rule-based creative tasks such as poetry, where clear guidelines exist they become impractical for evaluating broader forms of creativity. In such cases, it’s difficult to define and implement explicit rules, highlighting the need for more advanced and general reward techniques such as logprob-based rewards.

3.2.2 Logprob-Based Rewards

Logprob rewards evaluate the model’s internal confidence in the gold answer a , conditioned on the prompt p and its own reasoning trace r :

$$R = \log P(a \mid p, r)$$

Unlike rule-based rewards, which rely on hand-crafted criteria or task-specific evaluators, this signal is intrinsic to the model and can be computed across any domain where a gold answer is available.

To stabilize training, a baseline is computed as the mean reward across a batch:

$$b = \frac{1}{N} \sum_{i=1}^N \log P(a_i \mid p, r_i)$$

The advantage is then:

$$A = R - b$$

This relative advantage helps normalize reward signals across prompts of varying difficulty, focusing learning on outperforming the average trace within a group.

This entire setup encourages the model to internalize which reasoning paths increase confidence in correct answers, leading to more structured and faithful generation at inference time.

Two aspects make logprob-based rewards particularly powerful:

- **Continuity of feedback:** Unlike binary rule-based signals (e.g., format reward = 0 or 1), logprob values are continuous and differentiable. This enables more nuanced learning, where the model can improve by degrees even when far from the correct output. For example, a reasoning trace that moderately supports the correct answer receives partial credit, guiding the model in a meaningful direction.

- **Domain generality:** Because the reward only requires a prompt and a gold answer, it generalizes naturally to any task with a known target—whether classification, generation, or multi-modal alignment. This bypasses the need for designing custom evaluators or domain-specific heuristics, making it ideal for scaling reward learning across varied task types.

Together, these properties allow logprob-based GRPO to train models in a way that is more flexible and that can be more informative than rule-based approaches, particularly in subjective or under-specified domains. The goal of this research is to test the limits of logprob-based GRPO in a closed and open-ended task (See our [Results and Analysis](#) section).

3.3 Reward Normalization

In our setup, reward normalization was exclusively applied to logprob-based rewards. Because raw log-probabilities are negative and scale with answer length, we first applied **length normalization**, dividing the total log-probability by the number of tokens in the golden answer. This produces a more consistent signal across sequences of different lengths.

To further stabilize the training signal and highlight differences between outputs, we experimented with two additional batch-level normalization techniques:

- **Z-Score Normalization:** Mean-centering and scaling within a batch to reduce variance in the advantage signal.
- **Min-Max Scaling:** Rescaling length-normalized rewards to the $[0, 1]$ range for interpretability and dynamic contrast.

We found that combining length normalization with min-max scaling led to the most stable training behavior, especially in early stages when generations were highly variable.

These strategies helped mitigate verbosity bias and ensured that the optimization process did not favor longer or superficially different completions.

4 Experimental Setup

4.1 Task and Dataset Design

Experiments were carried out on contrasting tasks, one highly structured and rule-bound (math), and

the other open-ended and creative (poetry). This duality allows us to assess whether a domain-general reward signal can adaptively elicit reasoning across both constrained and subjective problem spaces.

Symbolic Math Task

Each prompt consists of a **set of four distinct integers** (e.g., {74, 5, 20, 88}) and a **target integer result** (e.g., 50). The model must construct a valid arithmetic expression using **each of the provided numbers exactly once**, producing a result that evaluates to the target. The solution must respect mathematical syntax and operator precedence.

This task was selected for its clarity of evaluation: correctness is binary and easily checked. However, the requirement for one-time use of each number imposes nontrivial combinatorial constraints, encouraging multi-step reasoning rather than memorized patterns. Structuring the task this way allows the model’s reasoning trace to play a meaningful role in whether it can assemble and justify a valid equation. Prompts used for this task can be found in the appendix B.1.

Poetic Continuation Task

In the poetry task, prompts consist of the beginning lines of a poem, with the model required to generate an appropriate stylistic continuation. Each instance also includes a **gold poem ending**, used both as a training signal and as a reward evaluation reference.

Unlike the math task, poetry completion is inherently **subjective and multi-dimensional**. To enable automated evaluation without costly human annotation, we adopted a **multi-metric reward design** capturing four linguistic properties:

- **Rhyme Accuracy:** Based on CMUdict to validate rhyme schemes.
- **Syllable Accuracy:** Counts syllables per line to match poetic form.
- **Form Classification:** Uses regex and heuristics to categorize form (e.g., limerick, haiku).
- **Semantic Similarity:** Measures embedding-based alignment with gold completion.

While imperfect, these metrics are fast to compute and provide sufficient resolution to evaluate reasoning in creative contexts. Prompts used for this task can be found in the appendix B.2.

4.2 Model Selection Strategy

We began with initial trials using two recent and capable language models, one of which was pre-trained with an emphasis on reasoning capabilities. This choice was intentional: by comparing a reasoning-oriented base model, Qwen3-1.7B, against a more general instruction-tuned model, we aimed to assess whether the performance gains from GRPO were dependent on the model’s prior reasoning ability. In particular, we were interested in whether reinforcement learning could elicit meaningful improvements even from a model that was already structured for reasoning.

- **Gemma-3-1B-IT** (DeepMind, 2024): A compact, 1-billion-parameter instruction-tuned model. While it has limited reasoning trace capabilities, it demonstrates strong general knowledge.
- **Qwen3-1.7B** (Team, 2024): A larger and more recent model from the Qwen family, which is known for being easily adaptable to reasoning tasks through fine-tuning.

Both models are relatively small, which makes it challenging for them to consistently follow detailed instructions and generate clear reasoning traces. This is a known issue but since we had limited training hardware this was the most efficient manner to conduct our experiments.

The smaller Gemma model showed significant difficulty in adhering to our required output structure (i.e., <think> and <answer> token boundaries). This formatting is crucial to compute rewards accurately, particularly for logprob-based methods, which condition the reward strictly on the reasoning trace. As such, we found that a model that **cannot reliably separate thinking from answering is fundamentally unsuitable** for GRPO-style reward shaping.

We therefore chose **Qwen3-1.7B**, which not only demonstrated better default reasoning but also allowed us to meaningfully measure the **marginal gains** provided by reinforcement learning.

4.3 Training Configuration

Precision and Memory Optimization

Since these models are relatively small, we **did not** apply any quantization techniques or use related libraries. Our goal was to assess the base performance of the models without introducing potential variability or degradation that quantization

might cause. Thus all training was performed with bfloat16 precision.

Sequence Constraints

Since GRPO heavily relies on model generations, shorter outputs lead to faster training. To reduce generation overhead and memory usage, we empirically set the `max_prompt_length` to 256 tokens and the `max_completion_length` to 1024 tokens. This trade-off allows us to explore a wider range of experimental settings, as long completions significantly increase GRPO runtime generation being the primary bottleneck.

Training Parameters

The following configuration was used throughout:

- Epochs: **2** for the math task, **5** for poetry
- Per-device batch size: **4**
- Gradient accumulation: **8 steps** $\rightarrow$ effective batch size = 32
- Learning rate: **5.0e-7**
- LR Scheduler: **cosine** with a **3% warmup ratio**
- Gradient checkpointing: **enabled**, with `use_reentrant: false` for compatibility with our model

Reward Computation

- **Number of generations per prompt:** 4
- **KL divergence penalty (β):** 0.001, lower than DeepSeekMath’s 0.04

Lowering the KL penalty allows the policy to deviate more from the reference model, which we deemed necessary to allow logprob-guided reasoning to manifest in creative domains.

4.4 Infrastructure and Implementation

Training was run on 2 GPUs (either A100 or H100) with 1 GPU being used for inference with the VLLM engine as a backend and 1 GPU for the GRPO training. VLLM offers significant speedups due to paged attention and KV-caching, but it does not expose log probabilities which is a major limitation for logprob-based reward computation. This forced us to decouple generation and evaluation and thus, prompt completions were generated using VLLM for efficiency and item rewards were computed on the training GPU using the standard transformers library.

4.5 Codebase and Customization

Custom GRPOTrainer. The default GRPO loss computes token-level advantages across the full output sequence, which is incompatible with our logprob-based reward design where rewards depend solely on the reasoning trace, not on the generated answer. To address this, we implemented a CustomGRPOTrainer that modifies the loss computation to exclude all tokens following the `<\think>` delimiter. This ensures the loss aligns strictly with the segment that influences the reward.

The modified GRPO objective is expressed as:

$$\mathcal{L}_{\text{GRPO}}(\theta) = \frac{1}{G} \sum_{i=1}^G \mathcal{L}_{\text{GRPO}}^{(i)}(\theta) \quad (1)$$

$$\mathcal{L}_{\text{GRPO}}^{(i)}(\theta) = \frac{1}{|o_i^{\leq t_{\text{end}}}|} \sum_{t=1}^{t_{\text{end}}} \ell_{i,t} - \beta D_{\text{KL}}[\pi_{\theta} \parallel \pi_{\text{ref}}] \quad (2)$$

$$\ell_{i,t} = \min \left[\frac{\pi_{\theta}(o_{i,t} \mid q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})} \cdot \hat{A}_{i,t}, g(\epsilon, \hat{A}_{i,t}) \right] \quad (3)$$

Here:

- $o_i^{\leq t_{\text{end}}}$ denotes the truncated output, stopping at the last reasoning token (i.e., before the `<\think>` tag ends).
- $\hat{A}_{i,t}$ is the token-level advantage, and $g(\epsilon, \cdot)$ is the clipping function.
- The final KL term penalizes divergence from a reference model to stabilize policy updates.

This design ensures that the model is updated solely on the parts of its output that contribute meaningfully to the reward, the reasoning trace. Training on tokens beyond t_{end} would introduce conflicting gradients, particularly when the generated answer disagrees with the gold answer despite a high log-probability reward.

Tooling and Infrastructure. For logging and tracking, we used both Weights & Biases (Wandb) and TensorBoard, enabling real-time monitoring of training dynamics and reproducibility. While our implementation supports multi-node training via DeepSpeed, we opted for single-node runs due to resource constraints. Nonetheless, the system is engineered to scale efficiently in larger distributed setups.

5 Results and Analysis

The main goal of our experiments was to evaluate how various reward functions and normalization strategies in GRPO influence the reasoning abilities of a pretrained language model, while simultaneously improving the quality of its answers. We will discuss the findings by analysing and discussing the results for the mathematical task first and then the creative task.

5.1 Mathematical reasoning

For mathematical tasks, where final answers can be verified using external tools, rule-based rewards are more appropriate. These give a reward of 1 for a correct result and 0 otherwise. In contrast, log-based rewards can assign high values to answers with a plausible thought process, even if the final answer is incorrect. As a result, we expect log-based rewards to underperform in math tasks, since they may encourage the model to settle for partially correct answers—undesirable when exact correctness is essential.

As shown in Figure 1, the rule-based rewards consistently perform better, achieving a global training reward exceeding 0.8, while the logprob-based rewards never surpass 0.6. Moreover, the training stability, in terms of global reward fluctuations, is slightly higher for rule-based rewards. These observations align with our intuition that rule-based rewards are better suited for mathematical tasks.



Figure 1: Global training reward curves for math: Rule-based (pink) vs. Logprob-based (blue).

To better understand what these differences mean in terms of model performance, we compare the final benchmark scores.

As shown in Figure 2, rule-based GRPO achieves a significantly higher score on equation correctness (0.612 vs. 0.289). However, logprob-based GRPO slightly outperforms on format adherence (0.313 vs. 0.285), reflecting its focus on structured reasoning. These results confirm that while

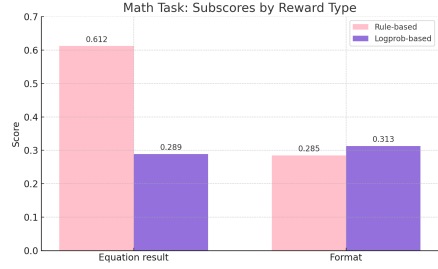


Figure 2: Math Task: Subscores comparing Rule-based and Logprob-based GRPO.

logprob guidance can enhance reasoning trace quality, it may not reliably guide models to exact solutions in tasks requiring binary correctness.

Training curves for both reward types for math task are shown in Appendix 7. Examples of samples before and after training can be found in the Appendix 5.

5.2 Creative writing and poetry

Creative writing is inherently subjective and difficult to evaluate with strict criteria. As such, we initially expected models to benefit more from **logprob-based rewards**, which offer flexible and continuous feedback. In contrast, rule-based rewards can penalize stylistically valid generations due to minor structural violations.

However, as shown in Figure 3, rule-based consistently outperforms logprob-based during training in terms of global reward. The rule-based curve (pink) rises steadily and stabilizes above 0.6, while the logprob-based variant (blue) remains more volatile and plateaus around 0.4. This suggests that logprob rewards, despite their generality, struggled to optimize the target signal effectively in this task setup.

To further understand model behavior, we compare subscores on the final benchmark (Figure 4). Surprisingly, rule-based GRPO outperformed logprob-based GRPO on every submetric, including rhyme accuracy, syllable accuracy, poetic form, and semantic similarity. The largest gap is observed in syllable accuracy (0.116 vs. 0.046), followed by form classification (0.091 vs. 0.059).

While this outcome contradicts our initial hypothesis, we believe logprob-based rewards still hold promise. In more complex or loosely structured creative tasks (e.g., story continuation, opinionated essays), they may provide richer signals than rigid rule checkers. Our current setup involved relatively strict evaluation metrics, even in poetry,

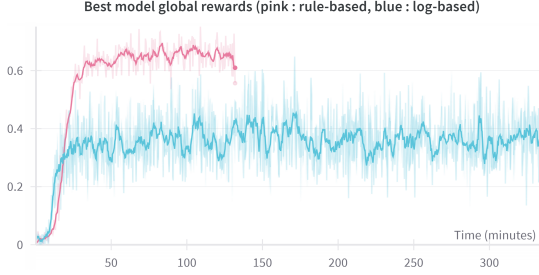


Figure 3: Global reward comparison on the poetry task: rule-based (pink) vs logprob-based (blue).

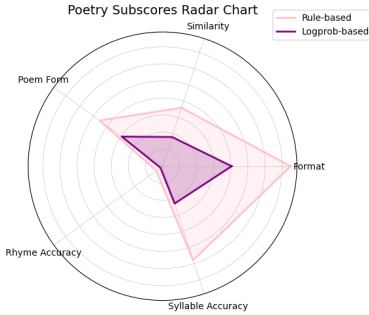


Figure 4: Poetry subscores radar chart comparing rule-based and logprob-based GRPO.

which may have favored the deterministic nature of rule-based rewards.

Training curves for both reward types for poetry task are shown in Appendix 8. Examples of samples before and after training can be found in the appendix 6.

5.3 A discussion about normalization in log based rewards

In our log-probability-based reward system, the log-probabilities of the golden (reference) answer are computed given the question and the model-generated thought process. The core idea is that log-probability measures confidence in the golden answer, thus a higher one should be awarded a better reward.

To ensure reliable reward computation, the model’s output must follow a consistent format that clearly separates the thought process from the final answer. This structure is essential for identifying and replacing the model’s generated answer with the golden one during reward calculation. Any generation that does not adhere to this format is assigned a reward of 0.

Once the format is validated, the final answer from each generation is removed and replaced with the golden answer. Then, the log-probabilities of

the golden answer are computed by conditioning on the question and the generated reasoning. The log-probability is then computed for the tokens in the golden answer, to assess the model’s confidence in the valid response.

However, since log-probabilities are inherently negative and decrease linearly with longer sequences, especially in tasks like poetry where the open-ended nature of the output leads to significant variation in answer length, this can introduce numerical instability when mapping them back to a $[0, 1]$ reward scale. To address this, we implemented a length normalization step: we average the total log-probability over the number of tokens in the golden answer. We then exponentiate this average to obtain a value between 0 and 1, making it more suitable for use as a reward.

In addition to this normalization, we take advantage of relatively large batch sizes (64 generations per reward computation) to apply further normalization strategies within the batch. Specifically, we experimented with both min-max normalization and z-score normalization on top of the length-normalized scores.

These additional normalization techniques serve as an adaptive reward mechanism: early in training, when most generations are poor, they help highlight the few high-quality generations. Later, when most generations become acceptable, this normalization sharpens the distinction between good and excellent outputs providing more informative reward signals throughout training.

We found that using min-max rewards on top of length normalization was the best setup, resulting in stable and increasing rewards over time.

For log-probability-based training in the poetry task, we tested four different setups, as illustrated in Figure 8f.

In the setup labeled min-max + basic-format, no length normalization was applied. Rewards were simply scaled to the $[0, 1]$ range using min-max normalization. While the log-probability component of the reward steadily increased over time, the format reward began to decline. This drop occurred because the model learned to exploit the reward structure: since the format adherence was weighted at only 10% of the total reward, the model began to ignore strict formatting constraints. Instead, it repeatedly generated multiple `<think>...</think>` blocks, which boosted the log-probability of the golden answer by aligning with the expected "thinking" phase—but without ever producing a mean-

ingful final answer. In essence, the model learned to "just think" without concluding.

To mitigate this, we introduced a constraint in the log-reward computation: only one `<think>` and `</think>` pair was allowed in the generation. This, combined with length normalization, aimed to enforce structure while preserving reward fidelity. However, this approach led to underwhelming training and rewards were frequently zero due to the exponentiation of highly negative averaged log-probabilities.

We then experimented with setups using z-score normalization and min-max normalization. Through these trials, we observed that penalizing the model with negative rewards, particularly in the challenging, open-ended poetry domain was not effective. In contrast, min-max normalization yielded more stable training dynamics, with both the format and log-probability rewards increasing consistently over time.

5.4 Benchmarks and models

We developed a custom benchmark suite using the lighteval framework to evaluate our models on mathematical and poetry tasks. All evaluations were conducted on dedicated test sets available on Hugging Face:Jeremmyyyyy/Math and schifferlearning/Poetry-Categorized. These benchmarks rely on predefined rules tailored to assess model performance according to our task-specific criteria. The table below lists each evaluated model alongside its Hugging Face ID and benchmark scores. The full codebase, including training scripts and evaluation tools, is available on GitHub: <https://github.com/theoschiff/TheElicitors>.

6 Conclusion

We explored reinforcement learning to elicit reasoning in LLMs using the DeepSeek-R1 framework with **logprob-based rewards**. Our **custom GRPO training loop** aligned the loss with the reward signal via token masking, focusing updates on the reasoning trace.

Our findings across math and poetry tasks highlight:

- **Rule-based rewards** consistently outperformed, even in creative tasks, due to their precise and verifiable feedback.
- **Logprob-based rewards**, while scoring

Task	Metric	Score
Baseline		
Math	Overall	0.009
Poetry	Overall	0.000
Rule-based GRPO		
Math	Overall	0.449
	Equation result	0.612
	Format	0.285
Poetry	Overall	0.091
	Format	0.150
	Similarity	0.072
	Poem form	0.091
	Rhyme accuracy	0.009
	Syllable accuracy	0.116
Logprob-based GRPO		
Math	Overall	0.301
	Equation result	0.289
	Format	0.313
Poetry	Overall	0.046
	Format	0.081
	Similarity	0.036
	Poem form	0.059
	Rhyme accuracy	0.003
	Syllable accuracy	0.046

Table 1: Final benchmark scores on math and poetry tasks.

lower, encouraged stylistic diversity and coherence—traits valuable in subjective domains.

- These rewards hold more promise in open-ended tasks like storytelling or dialogue, where rigid rules hinder creativity.
- **Normalization** was essential to stabilize logprob-based training, especially in high-variance tasks like poetry.

Overall, this work provides a practical framework for **GRPO reward shaping**, and demonstrates the potential of internal model confidence as a general-purpose training signal.

Future Work

- Apply logprob-based GRPO to multi-turn dialogue and long-form generation.
- Explore hybrid reward schemes combining rule- and logprob-based signals.
- Improve reward efficiency and scalability via VLLM-based computation.
- Investigate learned or adaptive normalization methods.

References

- Haolin Chen, Yihao Feng, Zuxin Liu, Weiran Yao, Akshara Prabhakar, Shelby Heinecke, Ricky Ho, Phil Mui, Silvio Savarese, Caiming Xiong, and Huan Wang. 2024. [Language models are hidden reasoners: Unlocking latent reasoning capabilities via self-rewarding](#). *Preprint*, arXiv:2411.04282.
- Google DeepMind. 2024. Gemma: Open models based on gemini research and technology. <https://ai.google.dev/gemma>.
- DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, and 181 others. 2025. [Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning](#). *Preprint*, arXiv:2501.12948.
- Alexander Gurung and Mirella Lapata. 2025. [Learning to reason for long-form story generation](#). *Preprint*, arXiv:2503.22828.
- Shibo Hao, Yi Gu, Haodi Ma, Joshua Jiahua Hong, Zhen Wang, Daisy Zhe Wang, and Zhiting Hu. 2023. [Reasoning with language model is planning with world model](#). *Preprint*, arXiv:2305.14992.
- Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2023. [Large language models are zero-shot reasoners](#). *Preprint*, arXiv:2205.11916.
- Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. 2022. [Training language models to follow instructions with human feedback](#). *Preprint*, arXiv:2203.02155.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. [Deepseekmath: Pushing the limits of mathematical reasoning in open language models](#). *Preprint*, arXiv:2402.03300.
- Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, Chunling Tang, Congcong Wang, Dehao Zhang, Enming Yuan, Enzhe Lu, Fengxiang Tang, Flood Sung, Guangda Wei, Guokun Lai, and 75 others. 2025. [Kimi k1.5: Scaling reinforcement learning with llms](#). *Preprint*, arXiv:2501.12599.
- Qwen Team. 2024. Qwen: A family of open-source large language models. <https://huggingface.co/Qwen/Qwen3-1.7B>.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. [Self-consistency improves chain of thought reasoning in language models](#). *Preprint*, arXiv:2203.11171.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2023. [Chain-of-thought prompting elicits reasoning in large language models](#). *Preprint*, arXiv:2201.11903.

A Team

Task	Team Member(s)
Data Collection and Preprocessing	Jeremy, Théo
Reward functions implementation	Aymeric, Lindsay, Théo, Jeremy
Normalization schemes implementation	Zeineb, Jeremy
Models Training	Lindsay, Jeremy, Théo
Benchmarks and Evaluation	Jeremy
Poster & Report	All team members

Table 2: Breakdown of tasks and the team members responsible for each.

B Prompts

B.1 Math task prompt

Math Prompt

System Prompt: You are a helpful assistant. You first think about the reasoning process in the mind and then provide the user with the answer.

Prompt Template: Using the numbers [numbers], create an equation that equals [target]. You can use basic arithmetic operations (+, -, *, /) one or multiple times but each number can only be used once. Show your work in `<think>` `</think>` tags. And return the final equation in `<answer>` `</answer>` tags, for example `<answer> (1 + 2) / 3 </answer>`. Think step by step inside `<think>` tags.`<think>` tags.

Assistant: Let me write an equation step by step. `<think>`

B.2 Poetry task system prompt

Poetry Prompt

System Prompt: You are a helpful assistant. You first think about the reasoning process in the mind and then provide the user with the answer.

Prompt Template: Using the author name : [author], the title : [title], the poem form : [form] and the start of an existing poem that is as follows : [poem_start], create a new ending for this poem. You need to take into account the form the rhymes, the syllables and theme to create a new realistic ending. Show your work in `<think>` `</think>` where you are allowed to think about structure, rhymes, etc. Return the final poem ending in `<answer>` `</answer>` tags. Think step by step inside `<think>` tags.

Assistant: Let me write a poem step by step. `<think>`

C Examples

D Training Curves

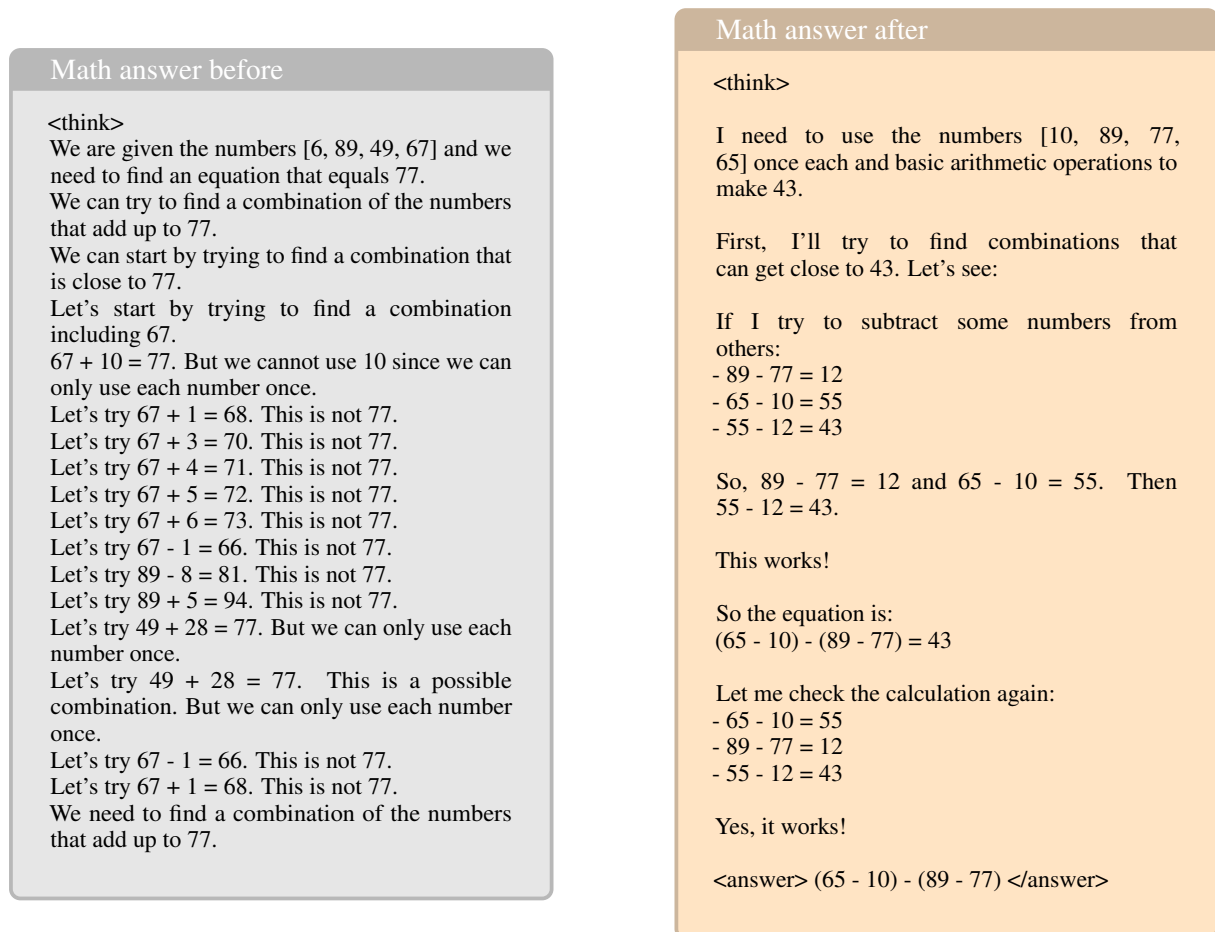


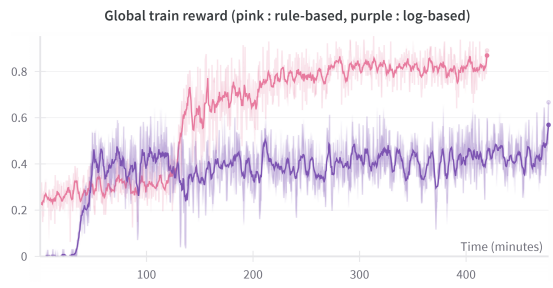
Figure 5: Base model VS finetuned model comparison on the rule based math training

Note that before training the model does provide the initial <think> tag and a reasonable thinking trace, but the strategy is very bad and frankly completely false. On the flip side, after training we can see that the model learnt to follow the thinking tags and has a significantly reasonable reasoning strategy. Most interestingly, we see that the model even checks its results, which is an interesting emergent behavior.

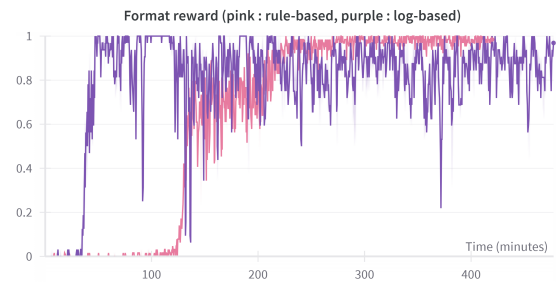


Figure 6: Base model VS finetuned model comparison on the rule based poetry

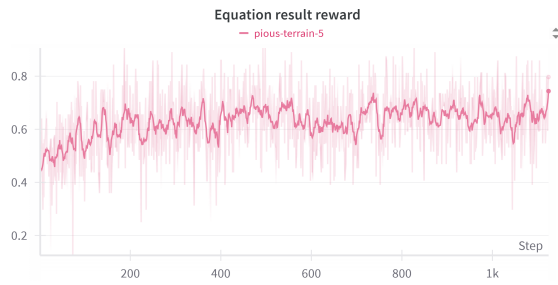
Note that before training, only provides the initial <think> tag, but doesn't satisfy the general reasoning format. On the flip side, after training we can see that the model learnt to follow the thinking tags and has a significantly reasonable step-by-step reasoning strategy.



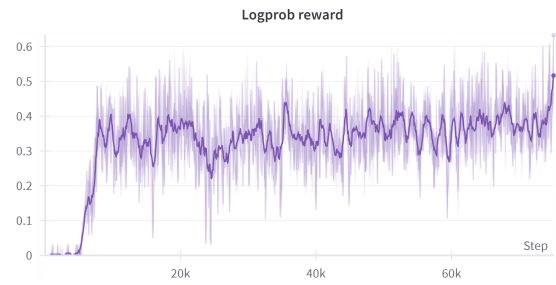
(a) Global reward comparison between rule-based and log-based training over time on same hardware



(b) Format reward comparison between rule-based and log-based training over time on same hardware

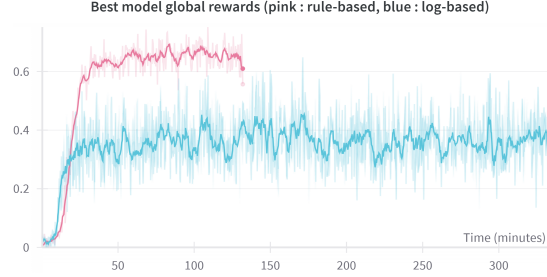


(c) Rule based equation result reward

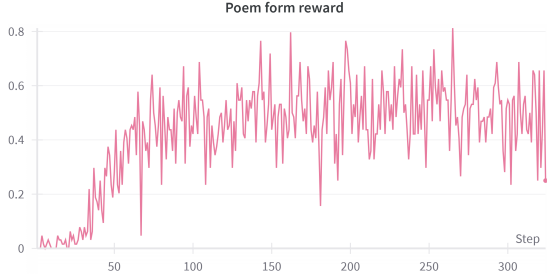


(d) Log-based reward

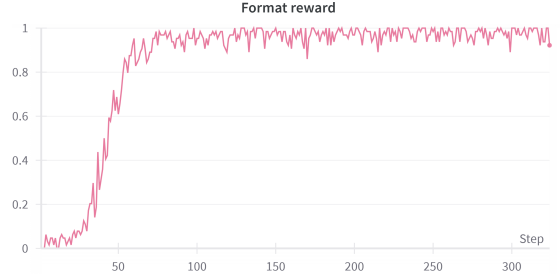
Figure 7: Training rewards on the math task with rule and log based rewards.



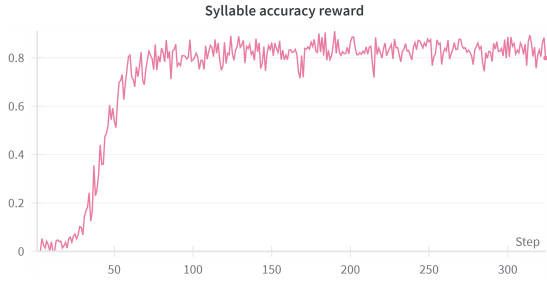
(a) Global reward comparison between rule-based and log-based training over time on same hardware



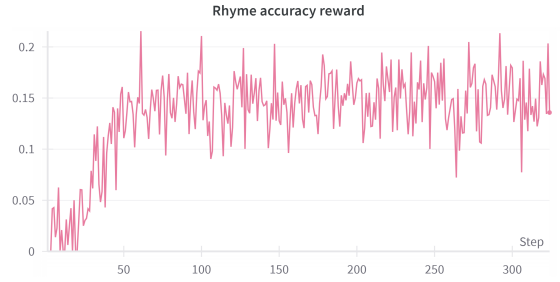
(b) Rule based poem form reward



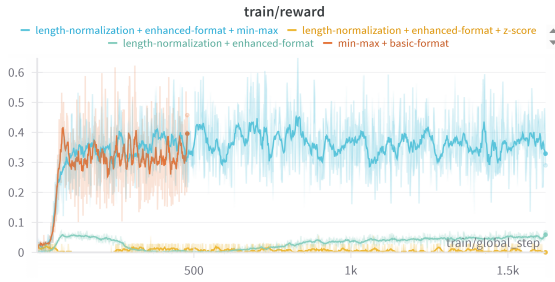
(c) Rule based poem format reward



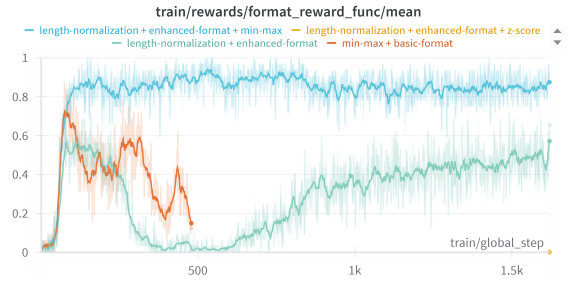
(d) Rule based poem syllable reward



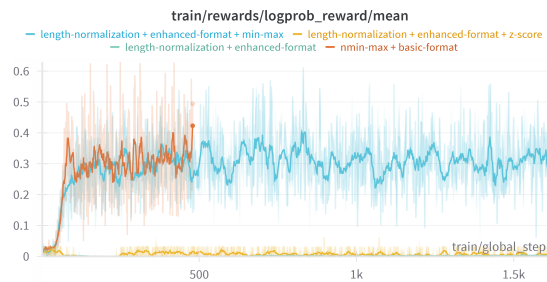
(e) Rule based rhyme reward



(f) Log-based global train reward



(g) Log-based format reward weighted as 10% of the total reward



(h) Log-based logprob reward weighted as 90% of the total reward